

Practice Online: DFT & FFT

Ten problems in the format of Online 4 (A1/A2, B1/B2)
five on Task A (big-integer multiplication), five on Task B (image convolution)

How to use this set

- Templates are in `practice/problems/`, full solutions in `practice/solutions/`. Each template has exactly three `TODO` blocks and a provided runner with an independent oracle that prints `verification: MATCH` or `MISMATCH`, exactly like the real sheets.
- Run a template *unmodified* first and watch which `TODO` it stops at. Then fill the three blocks. Then run it, and for Task B run it again with `--color`.
- Every file imports your own `transforms.py`, `bigmul.py`, `image_conv.py`. The only difference from a real online is the two `sys.path` lines at the top, so the files can run from the `practice/` folder instead of being copied next to the offline files. The real sheet's command is `python3 ...`; on your machine it is `python ....`
- Outputs land in `practice/outputs/<problem>/`. Every report must end with `verification: MATCH`; an error above 10^{-9} means a bug.
- Rules as in the real online: NumPy array arithmetic and the functions the template imports only; `numpy.fft`, SciPy transforms and library convolution routines are prohibited; Python big integers only in the verification line.
- Suggested order: fill `TODO 3` (dispatch/glue) first, then `TODO 1` (helper), then `TODO 2` (the core). Time yourself: 40 minutes each.

Solutions in plain language begin on page 12.

January 2026 CSE 220
Practice Online on DFT and FFT
Problem A1: Product of Three Integers

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

Template: `a1_triple_product.py`

The convolution theorem does not care how many factors there are. The digit array of $a \cdot b \cdot c$ is the linear convolution $a * b * c$, and in the frequency domain that is the pointwise product of *three* spectra. Three forward transforms, one triple product, one inverse transform.

You are given two operands from an offline input file and a third reproducible operand in `a1_triple_product.py`. Complete only the three blocks marked “TODO”. Do not modify `transforms.py`, `bigmul.py`, or any other offline file.

Mathematical model. Let a, b, c be little-endian limb arrays of lengths n, q, r in base B , with spectra A, B, C at a common length N . Their product has $n + q + r - 2$ coefficients and

$$P[k] = A[k] B[k] C[k], \quad p = \text{IDFT}\{P\}, \quad N \geq n + q + r - 2.$$

A coefficient of a triple product is a sum of roughly $n^2/2$ products of three limbs, so it can reach $(n^2/2)(B - 1)^3$. With $B = 10^4$ that exceeds the 2^{53} mantissa for the provided inputs; the template therefore fixes `TRIPLE_BASE_DIGITS = 2`, i.e. $B = 10^2$. Pass that base to both `to_limbs` and `from_limbs`. Signs stay outside the transform and multiply.

Tasks

1. Complete `choose_transform_length`. Return the minimum linear length for three factors, rounded to what the selected engine supports. [2 Marks]
2. Complete `multiply_three_transform`. Zero-pad, transform each operand once, multiply the three spectra, perform exactly one inverse transform, keep the real part, crop, round to `int64`. [5 Marks]
3. Complete `multiply_three`. Convert with the triple base, combine the signs, carry with the same base, return the product string. [3 Marks]

Running and verification.

```
python a1_triple_product.py --input inputs/3.txt --third-digits 2000 --engine fft
```

The runner writes `product.txt` and `report.txt` and compares against `int(a) * int(b) * int(c)` — the only permitted use of Python’s big integers. The report must show `verification: MATCH`. Negative operands (`inputs/1.txt`) must also work.

Problem A2: $ab + cd$ With One Inverse Transform

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

Template: `a2_sum_of_products.py`

The DFT is linear: the transform of a sum is the sum of the transforms. The sum of two products therefore needs four forward transforms but only *one* inverse transform — add the two product spectra first, invert once.

You are given four reproducible non-negative operands in `a2_sum_of_products.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. For limb arrays a, b, c, d with spectra A, B, C, D at a common length N ,

$$S[k] = A[k] B[k] + C[k] D[k], \quad s = \text{IDFT}\{S\}, \quad N \geq \max(n_a + n_b - 1, n_c + n_d - 1).$$

The coefficients of s are the un-carried digits of $ab + cd$; one carry sweep turns them into the decimal answer. All operands are non-negative: a signed sum can produce negative coefficients, which the offline’s `from_limbs` cannot carry, so a negative operand must raise `ValueError`.

Tasks

1. Complete `choose_transform_length`. One length must hold the longer of the two products; round to what the engine supports. [2 Marks]
2. Complete `sum_of_products_transform`. Pad and transform all four arrays, form $AB + CD$, perform exactly one inverse transform, keep the real part, crop to the longer linear length, round. [5 Marks]
3. Complete `sum_of_products`. Convert, reject negatives, convolve, carry, return the decimal string. [3 Marks]

Running and verification.

```
python a2_sum_of_products.py --digits 1500 --seed 11 --engine fft
```

The runner compares against `int(a)*int(b) + int(c)*int(d)`. The report must show `verification: MATCH`. Two inverse transforms is a wrong implementation even if it matches.

Problem A3: Circular Convolution and Wraparound

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

Template: `a3_circular_wraparound.py`

A length- N transform never computes linear convolution. It computes *circular* convolution of period N : anything that would land past slot $N - 1$ is added back onto slot $m \bmod N$. The padding rule exists only to make the two coincide. In this problem you compute the wrong thing on purpose and prove exactly how it is wrong.

You are given two reproducible operands in `a3_circular_wraparound.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. With linear convolution $\ell = a * b$ of length $n + q - 1$ and any $N \geq \max(n, q)$,

$$(a \circledast_N b)[m] = \sum_{k \equiv m \pmod{N}} \ell[k], \quad m = 0, \dots, N - 1.$$

The left side is what a length- N transform returns; the right side is the linear result *folded* onto N slots. They must agree exactly, integer for integer. The offline’s `multiply_transform` supplies ℓ .

Tasks

1. Complete `short_length`. Return $\max(n, q)$ rounded to what the engine supports — enough room for either input, deliberately too little for the product. [2 Marks]
2. Complete `circular_convolve`. Pad both arrays to exactly N , transform, multiply, inverse-transform, keep the real part, round. Return all N slots. [4 Marks]
3. Complete `fold`. Wrap a linear result onto N slots: `out[m % N] += linear[m]`. [4 Marks]

Running and verification.

```
python a3_circular_wraparound.py --digits 120 --seed 5 --engine fft
```

The runner checks $\max_m |\text{circular}[m] - \text{folded}[m]| = 0$ and that the padded route still equals `int(a) * int(b)`. The report also prints the plausible-looking wrong number the short transform produces, next to the correct one. It must show **verification: MATCH**.

Problem A4: Shift Detection by Cross-Correlation

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

Template: `a4_shift_correlation.py`

Correlation slides one sequence past another and measures how well they line up at each offset. It is convolution with a time-reversed sequence, and reversing a real sequence conjugates its spectrum — so correlation costs the same three transforms as multiplication, with one conjugate.

You are given a number a and $b = a \cdot B^s$ (the same limbs moved up by s positions) in `a4_shift_correlation.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. For limb arrays a (length n) and b (length q) the cross-correlation at lag k is

$$c[k] = \sum_i a[i] b[i + k], \quad \text{DFT}\{c\} = \overline{A} \cdot B, \quad N \geq n + q - 1,$$

with lag k stored at index $k \bmod N$ (negative lags sit at the end of the array). Because b is a shifted by s , the largest correlation occurs at $k = s$ and equals $\sum_i a[i]^2$, the energy of a .

Tasks

1. Complete `choose_transform_length`. Every lag needs its own slot: the linear-convolution rule applies unchanged. [2 Marks]
2. Complete `cross_correlate`. Pad, transform both, multiply $\overline{A}$ by B , perform one inverse transform, keep the real part, round to `int64`. Return the full length- N array and N . [5 Marks]
3. Complete `find_shift`. Correlate the limb magnitudes, locate the maximum, unwrap an index above $N/2$ to a negative lag, return the lag, the peak value and N . [3 Marks]

Running and verification.

```
python a4_shift_correlation.py --digits 400 --shift 7 --engine fft
```

The runner recomputes every lag with the defining double loop, checks the detected shift and checks that the peak equals the energy of a . The report must show **verification: MATCH**.

Problem A5: Fourth Power by Repeated Squaring

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

Template: `a5_fourth_power.py`

Squaring is multiplication with one operand, so it needs only *one* forward transform: $A \cdot A$. A fourth power is a square of a square. The catch is between the two squarings: the coefficients that come back from the first are not digits yet, and squaring un-carried coefficients breaks the mantissa bound.

You are given the first operand of an offline input file in `a5_fourth_power.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. For an n -limb array a with spectrum A ,

$$a^2 = \text{IDFT}\{A \cdot A\} \quad (2n - 1 \text{ coefficients}), \quad a^4 = (\text{carry}(a^2))^2.$$

The un-carried coefficients of a^2 reach $n(B - 1)^2 \approx 10^{11}$; squaring them directly would produce sums near 10^{25} , far past 2^{53} . Carrying first (`from_limbs`, then `to_limbs`) restores limbs below B , and the second squaring is safe. An even power is never negative.

Tasks

1. Complete `square_length`. A square has $2n - 1$ coefficients; round to what the engine supports. [2 Marks]
2. Complete `square_transform`. One forward transform, square the spectrum, one inverse, real part, crop, round. Two forward transforms is wrong. [4 Marks]
3. Complete `fourth_power`. Square, carry into a proper number, re-split into limbs, square again, carry, return the string together with both transform lengths and both limb counts. [4 Marks]

Running and verification.

```
python a5_fourth_power.py --input inputs/3.txt --engine fft
```

The runner compares against `int(a) ** 4`. The report must show `verification: MATCH`.

Problem B1: Shifting an Image With a Phase Ramp

Subsections: B1, B2

Total Marks: 10

Total Time: 40 Minutes

Template: `b1_spectral_shift.py`

Delaying a signal does not change how much of each frequency it contains, only where each oscillation sits — its phase. The shift theorem makes that exact: a delay of s samples multiplies the spectrum by a linear phase ramp. In two dimensions the ramps for rows and columns simply add in the exponent.

You are given an image and a shift in `b1_spectral_shift.py`. Complete only the three blocks marked “TODO”. Do not modify `transforms.py`, `image_conv.py`, or any other offline file.

Mathematical model. For an $H \times W$ plane with spectrum $F[u, v]$ and integer shifts (d_r, d_c) ,

$$F_{\text{shifted}}[u, v] = F[u, v] e^{-2\pi j \left(\frac{u d_r}{H} + \frac{v d_c}{W} \right)}, \quad u = 0..H-1, v = 0..W-1.$$

No padding is used, so the shift wraps around the edges: the result is a *circular* shift, which is exactly what `np.roll` computes. Negative shifts are allowed.

Tasks

1. Complete `phase_ramp`. Build the $H \times W$ complex multiplier above from the two index vectors. [3 Marks]
2. Complete `shift_plane`. Transform the plane, multiply by the ramp, perform exactly one inverse 2D transform, keep the real part. [4 Marks]
3. Complete `shift_image`. One plane for grayscale, three planes independently for RGB, original shape preserved. [3 Marks]

Running and verification.

```
python b1_spectral_shift.py --image images/skyline256.png --rows 37 --cols -21 --engine fft
python b1_spectral_shift.py --color
```

The runner compares against `np.roll(image, (rows, cols), axis=(0, 1))` and writes `shifted.png`, `comparison.png` and `report.txt`. The report must show **verification: MATCH**. Both grayscale and RGB inputs must work.

Problem B2: Unsharp Masking in the Frequency Domain

Subsections: B1, B2

Total Marks: 10

Total Time: 40 Minutes

Template: `b2_unsharp_mask.py`

A blur removes detail; subtracting the blur from the original isolates that detail; adding some of it back sharpens the picture. That is unsharp masking, and every step of it is a convolution or a sum, so the whole thing is one expression in the frequency domain.

You are given an image and a Gaussian kernel in `b2_unsharp_mask.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. With the padded spectra P (image) and G (kernel), the centred impulse spectrum Δ (provided helper) and a sharpening amount λ ,

$$y = x + \lambda(x - x * g) \iff Y = P(\Delta + \lambda(\Delta - G)) = P((1 + \lambda)\Delta - \lambda G).$$

Use one inverse transform and the usual linear-convolution crop starting at $(\lfloor K_r/2 \rfloor, \lfloor K_c/2 \rfloor)$. An $R \times C$ image and a $K_r \times K_c$ kernel need at least $(R + K_r - 1) \times (C + K_c - 1)$ samples; the radix-2 FFT needs each dimension raised to the next power of two.

Tasks

1. Complete `choose_transform_shape`. Minimum linear-convolution dimensions, then the engine’s restriction. [2 Marks]
2. Complete `sharpen_plane`. Pad, transform, form Y , perform exactly one inverse 2D transform, keep the real part, crop. [5 Marks]
3. Complete `sharpen_image`. Grayscale or RGB dispatch, shape preserved. [3 Marks]

Running and verification.

```
python b2_unsharp_mask.py --image images/skyline256.png --kernel-size 21 --amount 1.5 --  
engine fft  
python b2_unsharp_mask.py --color
```

The runner computes `image + amount * (image - convolve_image(image, kernel))` as an independent oracle. The report must show `verification: MATCH`.

Problem B3: Deblurring by Inverse Filtering

Subsections: B1, B2

Total Marks: 10

Total Time: 40 Minutes

Template: `b3_deblur.py`

If a blur is a pointwise multiplication in the frequency domain, undoing it is a pointwise division — as long as nothing is divided by zero. A circular blur by a small Gaussian can be undone exactly; the same idea applied to a large kernel amplifies rounding noise into garbage, and you will see why.

You are given a circularly blurred image and the kernel that blurred it in `b3_deblur.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. The offline’s circular path computes $Y = X \cdot G_w$, where G_w is the spectrum of the kernel wrapped around the origin (centre tap at index $(0,0)$). Then

$$X[u, v] = \begin{cases} Y[u, v]/G_w[u, v], & |G_w[u, v]| > \varepsilon, \\ 0, & \text{otherwise,} \end{cases} \quad x = \text{Re IDFT}\{X\}.$$

Rounding noise of size 10^{-16} in Y becomes $10^{-16}/|G_w|$ in X : a 5×5 Gaussian keeps $|G_w| \geq 4 \times 10^{-3}$ and restores the image to 10^{-14} ; a 31×31 Gaussian has bins near 10^{-50} .

Tasks

1. Complete `wrapped_kernel_spectrum`. Embed the kernel at the origin of a zero array of the plane’s shape, roll by $(-\lfloor K_r/2 \rfloor, -\lfloor K_c/2 \rfloor)$, transform. [3 Marks]
2. Complete `deblur_plane`. Divide only where $|G_w| > \varepsilon$, zero elsewhere, one inverse transform, real part. Never divide by a vanishing bin. [4 Marks]
3. Complete `deblur_image`. Grayscale or RGB dispatch, shape preserved. [3 Marks]

Running and verification.

```
python b3_deblur.py --image images/skyline256.png --kernel-size 5 --engine fft
python b3_deblur.py --color
```

The runner blurs with `convolve_image(..., circular=True)`, calls your `deblur` and compares with the original image. The report must show **verification: MATCH**. Afterwards try `--kernel-size 31` and read the “smallest $|G|$ bin” line of the report.

January 2026 CSE 220
Practice Online on DFT and FFT
Problem B4: Phase Correlation

Subsections: B1, B2

Total Marks: 10

Total Time: 40 Minutes

Template: `b4_phase_correlation.py`

Online B showed that phase carries the structure of an image. Phase correlation uses that: if two images differ only by a shift, their spectra differ only by a phase ramp, and dividing that ramp out of the cross-power spectrum leaves a pure impulse whose position *is* the shift.

You are given an image and its shifted copy in `b4_phase_correlation.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. For planes a and $b = \text{roll}(a, (d_r, d_c))$ with spectra A and B ,

$$R[u, v] = \frac{\overline{A[u, v]} B[u, v]}{|A[u, v]} |B[u, v]|} = e^{-2\pi j(\frac{u d_r}{H} + \frac{v d_c}{W})}, \quad r = \text{Re IDFT}\{R\} = \delta[m - d_r, n - d_c].$$

Bins with $|\overline{A}B| \leq \varepsilon$ must be set to 0, never divided. The peak of r at (m, n) gives the shift; an index above $H/2$ (or $W/2$) represents a negative shift $m - H$ (or $n - W$). For RGB, sum the three channel surfaces before locating the peak.

Tasks

1. Complete `unit_cross_power`. Form $\overline{A}B$, normalise reliable bins to unit magnitude, zero the rest, no division by zero. [3 Marks]
2. Complete `correlation_surface`. Transform both planes, take the unit cross-power spectrum, one inverse transform, real part. [4 Marks]
3. Complete `find_shift`. Grayscale or RGB dispatch, locate the peak, unwrap it, return `(rows, cols, surface)`. [3 Marks]

Running and verification.

```
python b4_phase_correlation.py --image images/skyline256.png --rows 37 --cols -21 --engine
fft
python b4_phase_correlation.py --color
```

The runner shifts with `np.roll`, checks the detected shift and checks that the surface is a unit impulse to 10^{-9} . The report must show `verification: MATCH`.

January 2026 CSE 220
Practice Online on DFT and FFT
Problem B5: Ideal Low-Pass / High-Pass Split

Subsections: B1, B2

Total Marks: 10

Total Time: 40 Minutes

Template: `b5_ideal_filter.py`

Low frequencies describe broad, slow variation; high frequencies describe edges and texture. An ideal filter keeps every bin inside a radius of zero frequency and discards the rest — but in a DFT array zero frequency is at index 0 and the negative frequencies live at the *top* of the array, so the distance must be measured with wrapped indices.

You are given an image and a radius in `b5_ideal_filter.py`. Complete only the three blocks marked “TODO”. Do not modify any offline file.

Mathematical model. Bin u of an H -point DFT has frequency $f_u = \min(u, H - u)$, likewise $f_v = \min(v, W - v)$. With

$$M[u, v] = \begin{cases} 1, & \sqrt{f_u^2 + f_v^2} \leq r \\ 0, & \text{otherwise} \end{cases}, \quad x_{\text{low}} = \text{Re IDFT}\{FM\}, \quad x_{\text{high}} = \text{Re IDFT}\{F(1 - M)\},$$

linearity gives $x_{\text{low}} + x_{\text{high}} = x$ exactly, and Parseval’s theorem says the fraction of energy kept is the same whether measured on pixels or on spectrum bins.

Tasks

1. Complete `radial_mask`. Wrapped frequency distance on both axes, Euclidean radius test, return `float64`. [3 Marks]
2. Complete `split_plane`. One forward transform, two masked inverse transforms, real parts, return `(low, high)`. [4 Marks]
3. Complete `split_image`. Grayscale returns the pair directly; RGB splits each channel, stacks the three lows and the three highs separately, returns both. [3 Marks]

Running and verification.

```
python b5_ideal_filter.py --image images/skyline256.png --radius 20 --engine fft
python b5_ideal_filter.py --color
```

The runner checks $\max |x_{\text{low}} + x_{\text{high}} - x|$, compares your low-pass with one built from an independently constructed (centred, then rolled) mask, and checks Parseval. The report must show **verification: MATCH**.

Solutions in plain language

Every problem here is the same five-step pipeline you already wrote in the offline: *pad* $\rightarrow$ *transform* $\rightarrow$ *do something simple to the spectrum* $\rightarrow$ *inverse transform once* $\rightarrow$ *real part, crop, round*. What changes is only the “something simple”. Full code is in `practice/solutions/`; below is the idea behind each, written for a first-time reader.

The three facts everything rests on

1. **Convolution becomes multiplication.** The DFT rewrites a signal as a sum of complex exponentials. Convolution scales each exponential and never mixes it with its neighbours, so after the transform the whole tangled sum turns into N independent multiplications. Hence $a * b \leftrightarrow A \cdot B$, and with more factors $a * b * c \leftrightarrow A \cdot B \cdot C$.
2. **The DFT is linear.** $\text{DFT}\{x + y\} = X + Y$ and $\text{DFT}\{\lambda x\} = \lambda X$. So any expression built from sums and convolutions can be assembled entirely in the frequency domain and brought back with a single inverse transform.
3. **A shift is a phase ramp.** Delaying $x[n]$ by s samples turns $X[u]$ into $X[u] e^{-2\pi j u s / N}$: same magnitudes, rotated phases. Reversing a real sequence in time conjugates its spectrum.

And one rule of engineering: a length- N transform computes *circular* convolution, so pad to at least the linear length or the tail wraps onto the head.

A1 — product of three integers

Idea. Fact 1 with three factors. Transform a, b, c once each, multiply the three spectra bin by bin, invert once.

Why the base drops to 10^2 . Every coefficient of the result comes back as a floating-point number that you round. Rounding only recovers the correct integer if the integer was representable: doubles hold integers exactly up to $2^{53} \approx 9 \times 10^{15}$. A coefficient of abc adds up about $n^2/2$ terms, each a product of three limbs, so it can reach $(n^2/2)(B-1)^3$. With $B = 10^4$ and a thousand limbs that is 10^{18} — too big. With $B = 10^2$ it is 10^{12} — fine. This is the same argument the offline made for choosing 10^4 over 10^9 , one factor further.

Steps. TODO 1: `need = n + q + r - 2`, power of two unless the engine is **arbitrary**. TODO 2: three zero-padded arrays, three `engine.transform`, one product, one `engine.inverse`, `.real`, crop to `need, np rint`. TODO 3: `to_limbs(text, 2)` three times, product of the three signs, `from_limbs(sign, coeffs, 2)`.

Pitfall. Forgetting to pass the base to `from_limbs`: the carry sweep then uses 10^4 on base- 10^2 limbs and prints a wrong number with the right length.

A2 — $ab + cd$ with one inverse

Idea. Fact 2. $\text{DFT}\{a * b + c * d\} = AB + CD$. Four forward transforms, one addition, one inverse. Doing two inverses and adding the results also gives the right number, but it misses the point of the problem and is marked as such.

Why one N is enough. Both products must fit without wrapping, so N must be at least the longer of the two linear lengths; padding the shorter product with extra zeros costs nothing.

Why non-negative only. ab and cd are both non-negative, so their sum’s coefficients are non-negative and the carry sweep works. If one product were negative, some coefficients would be negative

and `from_limbs` would loop forever on a negative carry — hence the `ValueError`.

Steps. TODO 1: `max(1a+1b-1, 1c+1d-1)`, engine rounding. TODO 2: pad four, transform four, `A*B + C*D`, one inverse. TODO 3: convert, check signs, convolve, `from_limbs(1, coeffs)`.

A3 — circular convolution and wraparound

Idea. The transform of length N “thinks” the world is a circle of N slots. A coefficient that should land in slot $m \geq N$ lands in slot $m \bmod N$ instead, on top of whatever was already there. That is not an approximation — it is exact arithmetic, just of the wrong quantity. The problem makes you compute both sides of the identity and show they are equal.

Steps. TODO 1: `max(1a, 1b)`, engine rounding — a length that is big enough for the inputs but not for the product. TODO 2: identical to `multiply_transform` except that N is given and nothing is cropped. TODO 3: a loop adding `linear[m]` into `out[m % N]`.

What to notice in the report. The wrapped “product” has fewer digits and a completely different start; nothing crashed and no warning appeared. This is the silent failure the padding rule prevents.

A4 — shift detection by cross-correlation

Idea. Correlation $c[k] = \sum_i a[i] b[i+k]$ asks “how well does b match a when slid by k ?” It is convolution with a reversed. Fact 3: reversing a real sequence conjugates its spectrum, so $\text{DFT}\{c\} = \overline{A} B$. One `np.conj` is the entire difference from multiplication.

Why the peak is at s . Appending $4s$ decimal zeros multiplies by $10^{4s} = B^s$, which moves every limb up s places. Sliding by exactly s lines each limb up with itself, and the sum of squares is as large as it can be. Every other lag mixes different limbs and comes out smaller.

Where negative lags live. There are $n + q - 1$ possible lags. Lag $k \geq 0$ sits at index k ; lag $k < 0$ sits at index $N + k$. If the peak index is above $N/2$ it is really a negative lag, so subtract N .

Steps. TODO 1: `1a + 1b - 1`, engine rounding. TODO 2: pad, `np.conj(engine.transform(fa)) * engine.transform(fb)`, inverse, `.real`, `np rint`. TODO 3: `np.argmax`, `unwrap`, return.

A5 — fourth power by repeated squaring

Idea. $a^2 = \text{IDFT}\{A \cdot A\}$ needs one forward transform, not two. $a^4 = (a^2)^2$.

Why carry in between. After the first squaring, the coefficients are sums like $\sum a[i] a[m - i]$, up to $n(B - 1)^2 \approx 10^{11}$ each — far larger than a limb. They are a correct answer, just not written in base B . Squaring them again would produce coefficients around 10^{25} , past the mantissa, and rounding would return wrong integers. Running the carry sweep (`from_limbs`) and re-splitting (`to_limbs`) rewrites a^2 as proper limbs below B , and then the second squaring is exactly as safe as the first.

Steps. TODO 1: `2n - 1`, engine rounding. TODO 2: one `engine.transform`, `A*A`, inverse, crop, round. TODO 3: `square, to_limbs(from_limbs(1, square))`, square again, `from_limbs`.

B1 — shifting an image with a phase ramp

Idea. Fact 3 in two dimensions. Moving the picture down by d_r rows and right by d_c columns multiplies bin (u, v) by $e^{-2\pi j(ud_r/H + vd_c/W)}$. Magnitudes are untouched — the picture contains the same frequencies — only phases turn.

Why it wraps. The DFT treats the image as periodic, so content pushed off the bottom edge re-enters at the top. That is exactly `np.roll`, which is why it is the oracle. No padding, no crop.

Steps. TODO 1: two index vectors `u[:, None]` and `v[None, :]`, one `np.exp`. TODO 2: `transform_2d`, multiply, `inverse_2d`, `.real`. TODO 3: the standard dispatch.

Pitfall. A plus sign in the exponent shifts the other way; the oracle catches it immediately.

B2 — unsharp masking

Idea. $\text{Detail} = x - x * g$ (original minus blur). $\text{Sharpened} = x + \lambda \cdot \text{detail}$. In the frequency domain, $x * g$ is PG and the unblurred x is $P\Delta$ — Δ is the spectrum of an impulse sitting at the kernel’s centre, so that after the same crop it lands exactly on the original pixels (that is the whole reason Online A provided it). Then $Y = P\Delta + \lambda(P\Delta - PG) = P(\Delta + \lambda(\Delta - G))$, one inverse transform, one crop.

Steps. TODO 1: exactly Online A’s `choose_transform_shape`. TODO 2: pad plane and kernel, two `transform_2d`, the formula, one `inverse_2d`, `.real`, crop at `(kh//2, kw//2)`. TODO 3: the standard dispatch.

Pitfall. Using 1 instead of Δ for “the original”: a plain 1 is an impulse at the corner, not at the kernel centre, so the original comes out displaced by half a kernel relative to the blur.

B3 — deblurring by inverse filtering

Idea. A circular blur is $Y = XG_w$ bin by bin, so $X = Y/G_w$ bin by bin. Division is only allowed where G_w is not (numerically) zero.

Why the kernel must be wrapped. The offline’s circular path puts the kernel’s centre tap at index $(0, 0)$ with `np.roll`. The blur you are undoing was made that way, so the spectrum you divide by must be of the identically placed kernel, or every pixel comes back shifted by half a kernel.

Why big kernels cannot be inverted. A blur kills high frequencies: $|G_w|$ is tiny there. Dividing by a tiny number multiplies whatever is in Y at that bin — including 10^{-16} rounding noise — by a huge factor. For a 5×5 Gaussian the smallest bin is about 4×10^{-3} , so noise grows $250\times$ and stays invisible. For 31×31 the smallest bins are around 10^{-50} ; the ε mask zeros those, but bins just above ε still amplify by 10^{12} . This is the mantissa argument again, from the other side.

Steps. TODO 1: zeros of the plane’s shape, kernel at top-left, `np.roll` by `(-(kh//2), -(kw//2))`, `transform_2d`. TODO 2: mask $|G_w| > \varepsilon$, divide at the mask only, zeros elsewhere, one inverse, `.real`. TODO 3: the standard dispatch.

B4 — phase correlation

Idea. If b is a shifted, then by Fact 3 $B = A \cdot \text{ramp}$. Multiply by $\overline{A}$: $\overline{A}B = |A|^2 \cdot \text{ramp}$. Divide by the magnitude: only the ramp is left. The inverse transform of a pure ramp is a single impulse of height 1 at the shift — an entire image collapses to one bright pixel whose position is the answer.

Why normalise. Without the division the surface is $\text{IDFT}\{|A|^2 \text{ramp}\}$ = the autocorrelation of a shifted — a broad blob, still peaked at the right place but far less sharp. Keeping only phase is what makes the peak an impulse; that is the same lesson as Online B.

Steps. TODO 1: the `unit_phase` pattern from Online B applied to `np.conj(A) * B`, with zeros (not ones) for unreliable bins. TODO 2: two `transform_2d`, TODO 1, one `inverse_2d`, `.real`. TODO 3: sum the channel surfaces for RGB, `np.argmax + np.unravel_index`, subtract H or W from an index above the half.

Pitfall. $A\overline{B}$ instead of $\overline{A}B$ puts the peak at $(-d_r, -d_c)$.

B5 — ideal low-pass / high-pass split

Idea. Multiply the spectrum by a mask that is 1 near zero frequency and 0 far from it: low-pass. The complementary mask $1 - M$: high-pass. By Fact 2, $FM + F(1 - M) = F$, so the two parts add back to the original exactly.

Why wrapped distances. In a DFT array bin 0 is zero frequency, bin 1 is the slowest oscillation, and bin $H - 1$ is *also* the slowest oscillation (going the other way). The frequency of bin u is therefore $\min(u, H - u)$, not u . Measuring plain distance from index $(0, 0)$ would treat the top-right corner as high frequency when it is low.

Why Parseval holds. The DFT is an orthogonal change of basis (up to the factor N), so it preserves lengths: $\sum |x|^2 = \frac{1}{N} \sum |X|^2$. The fraction of energy inside the mask is the same whether you sum pixels of x_{low} or bins of FM ; the runner checks that to 10^{-9} .

Steps. TODO 1: `np.minimum(u, H - u)`, same for columns, outer Euclidean distance, `<= radius`, `.astype(float64)`. TODO 2: one `transform_2d`, two `inverse_2d`, `.real` on both. TODO 3: the pair-returning dispatch — collect (low, high) per channel, then `np.stack` the lows and the highs separately.

Pitfall. Returning a bool mask: `spectrum * mask` with a bool array works, but `1 - mask` on a bool array does not mean what you think. Cast to `float64`.

The dispatch you will write every single time

```
if image.ndim == 2:
    return f(image, ...)
if image.ndim == 3 and image.shape[2] == 3:
    planes = [f(image[:, :, c], ...) for c in range(image.shape[2])]
    return np.stack(planes, axis=-1)
raise ValueError("images must be grayscale or RGB")
```

When `f` returns a pair, replace the last two lines of the RGB branch by `firsts, seconds = zip(*pairs)` and stack each with `axis=-1`. Type it from memory before the online starts.